

กลุ่มที่4 Expression Processor: Infix และ Postfix

1.วิเคราะห์ปัญหา:

โจทย์: รับนิพจน์คณิตศาสตร์แบบ Infix เช่น “3 + 4 * 2” เข้ามา ตรวจสอบว่าถูกไวยากรณ์หรือไม่ถ้าถูกต้องให้ แปลงเป็น Postfix แล้วคำนวณผลลัพธ์ออกมา

Input:

- String นิพจน์ที่มี Operator + - */ และวงเล็บ
- ตัวเลขเป็นจำนวนเต็มบวก มีได้หลายหลัก เช่น 123
- มีช่องว่างคั่นระหว่าง token ได้

Output:

- ถ้านิพจน์ถูกต้อง: Postfix Expression และผลลัพธ์ตัวเลข
- ถ้าผิด: แจ้ง error ว่าผิดตรงไหน

ข้อจำกัด:

- รองรับเฉพาะ + - * / กับวงเล็บกลมเท่านั้น
- เลขเป็นจำนวนเต็มบวกล้วน ไม่มีทศนิยม
- ห้ามหารด้วย 0

ข้อสมมติฐาน:

- input เป็น string ที่อ่านทีละ token ได้ชัดเจน
- operator มีpriority ตามหลักคณิตศาสตร์ปกติ: * / มาก่อน + -

กรณีปกติ:

- $3 + 4 * 2$ (คำนวณตาม Priority ปกติ)
- $(3 + 4) * 2$ (วงเล็บเปลี่ยนลำดับ)

กรณีขอบเขต:

- นิพจน์ว่าง
- นิพจน์ที่มีตัวเลขตัวเดียวเช่น 5
- วงเล็บซ้อนกันหลายชั้น เช่น $((8 + 2) * 5)$

กรณีที่ทำให้เกิด Error ได้:

- วงเล็บไม่ครบคู่ เช่น $(3 + 4$
- Operator ซ้อนกันโดยไม่มีoperand คั่น เช่น $3 + * 4$
- หารด้วยศูนย์เช่น $10 / (5 - 5)$

Operation ที่เกิดขึ้นบ่อยที่สุด:

- push/pop operand และ operator ระหว่างสแกนแต่ละ token

Operation ที่อาจใช้เวลามากที่สุด:

- ตอนเจอวงเล็บปิด “)” ต้อง pop operator ออกมาคำนวณจนกว่าจะเจอวงเล็บเปิด “(“

2. การออกแบบอัลกอริทึม:

Algorithm A: Infix to Postfix แล้ว Evaluate แยกอีกที

- คือแปลง Infix เป็น Postfix ก่อนทั้งหมด แล้วค่อยนำ Postfix ไปคำนวณผลลัพธ์แยกกันอีกขั้นตอน

CLASS AlgorithmA

// ฟังก์ชันหลักในการประมวลผลนิพจน์

FUNCTION evaluate(expression: String) -> ExpressionResult

TRY

 startTime = System.nanoTime()

 counter = NEW OperationCounter()

 tokens = Tokenizer.tokenize(expression)

 IF tokens.isEmpty() THEN

 THROW Exception("รูปแบบผิดพลาด ไม่ถูกต้อง")

 END IF

 postfixList = infixToPostfix(tokens, counter)

 resultValue = evaluatePostfix(postfixList, counter)

 postfixString = String.join(" ", postfixList)

 executionTime = System.nanoTime() - startTime

 RETURN ExpressionResult.success(resultValue, postfixString, executionTime, counter)

CATCH Exception e

 executionTime = System.nanoTime() - startTime

 RETURN ExpressionResult.error(e.getMessage(), executionTime, counter)

END TRY

END FUNCTION

```

// 1. แปลง Infix List เป็น Postfix List
FUNCTION infixToPostfix(tokens: List<String>, counter:
    OperationCounter) -> List<String> postfixList = NEW
    List()
    operatorStack = NEW Stack()
    FOR EACH token IN tokens DO
        counter.incrementLoop()

        IF isNumber(token) THEN
            postfixList.add(token)

        ELSE IF token == "(" THEN
            operatorStack.push(token)
            counter.incrementPush()

        ELSE IF token == ")" THEN
            WHILE NOT operatorStack.isEmpty() AND operatorStack.peek() != "(" DO
                counter.incrementLoop()
                postfixList.add(operatorStack.pop())
                counter.incrementPop()
            END WHILE

            IF operatorStack.isEmpty() THEN
                THROW Exception("วงเล็บไม่จับคู่กัน")
            END IF

            operatorStack.pop() // ถอด "(" ออก
            counter.incrementPop()

        ELSE IF isOperator(token) THEN
            WHILE NOT operatorStack.isEmpty() AND operatorStack.peek() != "("
            AND priority(operatorStack.peek()) >= priority(token) DO
                counter.incrementLoop()

```

```
        counter.incrementComparison()
        postfixList.add(operatorStack.pop())
        counter.incrementPop()
    END WHILE
```

```
        operatorStack.push(token)
        counter.incrementPush()
    ELSE
        THROW Exception("token ไม่ถูกต้อง: " + token)
    END IF
END FOR
```

// ย้ายตัวดำเนินการที่เหลือใน Stack ไปไว้ใน Postfix List

```
WHILE NOT operatorStack.isEmpty() DO
    counter.incrementLoop()
    topToken = operatorStack.pop()
    counter.incrementPop()
```

```
    IF topToken == "(" THEN
        THROW Exception("วงเล็บไม่จับคู่กัน")
    END IF
```

```
    postfixList.add(topToken)
END WHILE
```

```
RETURN postfixList
END FUNCTION
```

// 2. คำนวณผลลัพธ์จาก Postfix List

```
FUNCTION evaluatePostfix(postfixList: List<String>,
    counter: OperationCounter) -> Double
    operandStack = NEW Stack()
```

```
    FOR EACH token IN postfixList DO
        counter.incrementLoop()
```

```

IF isNumber(token) THEN
    operandStack.push(parseDouble(token))
    counter.incrementPush()

ELSE IF isOperator(token) THEN
    IF operandStack.size() < 2 THEN
        THROW Exception("รูปแบบผิดพลาด: ตัวเลขไม่
        เพียงพอสำหรับเครื่องหมาย " + token) END IF
    b = operandStack.pop()
    a = operandStack.pop()
    counter.incrementPop()
    counter.incrementPop()

    result = calculate(a, token, b)
    operandStack.push(result)
    counter.incrementPush()
END IF
END FOR

IF operandStack.size() != 1 THEN
    THROW Exception("รูปแบบผิดพลาด: ลำดับ
    เครื่องหมายหรือตัวเลขไม่ถูกต้อง") END IF

RETURN operandStack.pop()
END FUNCTION

// ฟังก์ชันช่วยประมวลผลทางคณิตศาสตร์
FUNCTION calculate(a: Double, op: String,
b: Double) -> Double SWITCH op DO
    CASE "+": RETURN a + b
    CASE "-": RETURN a - b
    CASE "*": RETURN a * b
    CASE "/":
        IF b == 0 THEN

```

```

        THROW ArithmeticException("หารด้วยศูนย์")
    END IF
    RETURN a / b
DEFAULT:
    THROW Exception("operator ไม่ถูกต้อง: " + op)
END SWITCH
END FUNCTION

```

```

// กำหนดลำดับความสำคัญของตัวดำเนินการ
FUNCTION priority(op: String) -> Integer
    IF op == "+" OR op == "-" THEN RETURN 1
    IF op == "*" OR op == "/" THEN RETURN 2
    RETURN 0
END FUNCTION

```

```

END CLASS

```

```

Expression Processor:
1. Algorithm A (Infix -> Postfix -> Evaluate)
2. Algorithm B (Direct Infix Evaluate)
0. ออกจากโปรแกรม
เลือกเมนู: 1
ป้อนนิพจน์ (Infix): 3 + 4 * 2 / (1 - 5)
--- Algorithm A ---
Postfix: 3 4 2 * 1 5 - / +
ผลลัพธ์: 1.0
เวลาที่ใช้: 549600 ns
จำนวน Operation: Push=14, Pop=14, Comparisons=6, Loops: 20

```

ข้อดี

- แยกขั้นตอนการแปลงรูปแบบนิพจน์กับการคำนวณค่าออกจากกันอย่างชัดเจน ทำให้การไล่โค้ด ตรวจสอบข้อผิดพลาด
- ผลลัพธ์Postfix List ที่ได้จากการแปลงสามารถนำไปใช้คำนวณซ้ำหลายๆ รอบได้ทันทีโดยไม่ต้องเสียเวลาแปลง นิพจน์Infix ใหม่

- จัดการวงเล็บและลำดับเครื่องหมายที่แม่นยำ การแปลงเป็น Postfix ช่วยขจัดความสับสนเรื่องลำดับความสำคัญ (Precedence) และวงเล็บซ้อนก่อนที่จะเริ่มคำนวณจริง

ข้อจำกัด

- ต้องวนอ่านข้อมูลทั้งหมด 2 รอบเต็ม (รอบแรกเพื่อสร้าง Postfix List และรอบสองเพื่อคำนวณผลลัพธ์ จาก Postfix)
- สิ้นเปลืองพื้นที่ความจำเพิ่มเติม จำเป็นต้องสร้างและถือโครงสร้างข้อมูล Postfix List (ArrayList) ค้างไว้ในระบบเพิ่มเติม

Time Complexity

- $O(n)$

Space Complexity

- $O(n)$

Algorithm B: ประเมิน Infix ตรงๆ ด้วย 2 Stacks

- คือ ใช้Operand Stack และ Operator Stack ประเมินค่านิพจน์Infix ไปพร้อมกันในรอบเดียว โดยไม่ต้อง แปลงเป็น Postfix ก่อน

CLASS AlgorithmB

```
// ฟังก์ชันหลักในการประมวลผลนิพจน์แบบ Infix โดยตรง
FUNCTION evaluate(expression: String) -> ExpressionResult
    TRY
        startTime = System.nanoTime()
        counter = NEW OperationCounter()
        tokens = Tokenizer.tokenize(expression)
```



```
IF tokens.isEmpty() THEN
    THROW Exception("รูปแบบผิดพลาด ไม่ถูกต้อง")
END IF
```

```
resultValue = evaluateInfix(tokens, counter)
executionTime = System.nanoTime() - startTime
```

```
// Algorithm B ประมวลผลแบบ Direct Infix จึงไม่มีPostfix String
RETURN ExpressionResult.success(resultValue, "", executionTime, counter)
CATCH Exception e
    executionTime = System.nanoTime() - startTime
    RETURN
ExpressionResult.error(e.getMessage(),
    executionTime, counter) END TRY
END FUNCTION
```

```
// 1. คำนวณนิพจน์Infix ด้วย Operands Stack และ
Operators Stack FUNCTION evaluateInfix(tokens:
List<String>, counter: OperationCounter) -> Double
operandStack = NEW Stack<Double>()
operatorStack = NEW Stack<String>()
```

```
FOR EACH token IN tokens DO
    counter.incrementLoop()
```

```
IF isNumber(token) THEN
    operandStack.push(parseDouble(token))
    counter.incrementPush()
```

```
ELSE IF token == "(" THEN
    operatorStack.push(token)
    counter.incrementPush()
```

```
ELSE IF token == ")" THEN
```

```

WHILE NOT operatorStack.isEmpty() AND
    operatorStack.peek() != "(" DO
    counter.incrementLoop()
    applyTopOperator(operandStack, operatorStack, counter)
END WHILE

IF operatorStack.isEmpty() THEN
    THROW Exception("วงเล็บไม่จับคู่กัน")
END IF

operatorStack.pop() // ถอด "(" ออกจาก Stack
counter.incrementPop()

ELSE IF isOperator(token) THEN
    WHILE NOT operatorStack.isEmpty() AND operatorStack.peek() != "("
AND priority(operatorStack.peek()) >= priority(token) DO
        counter.incrementLoop()
        counter.incrementComparison()
        applyTopOperator(operandStack, operatorStack, counter)
    END WHILE

    operatorStack.push(token)
    counter.incrementPush()

ELSE
    THROW Exception("token ไม่ถูกต้อง: " + token)
END IF
END FOR

// ประมวลผลตัวดำเนินการที่เหลือใน Operator Stack
WHILE NOT operatorStack.isEmpty() DO
    counter.incrementLoop()

    IF operatorStack.peek() == "(" THEN
        THROW Exception("วงเล็บไม่จับคู่กัน")
    
```

END IF

 applyTopOperator(operandStack, operatorStack, counter)
END WHILE

IF operandStack.size() != 1 THEN
 THROW Exception("รูปแบบผิดพลาด: ลำดับเครื่องหมายหรือตัวเลขไม่ถูกต้อง")
END IF

 RETURN operandStack.pop()
END FUNCTION

// 2. ฟังก์ชันดึงตัวดำเนินการและตัวเลขส่วนบนสุดของ Stack มาคำนวณ

FUNCTION applyTopOperator(operandStack: Stack<Double>, operatorStack:
 Stack<String>, counter: OperationCounter) IF operatorStack.isEmpty() THEN
 THROW Exception("รูปแบบผิดพลาด: เครื่องหมายไม่สมบูรณ์")
END IF

IF operandStack.size() < 2 THEN
 op = operatorStack.peek()
 THROW Exception("รูปแบบผิดพลาด: ตัวเลขไม่
เพียงพอสําหรับเครื่องหมาย " + op) END IF

op = operatorStack.pop()
counter.incrementPop()

b = operandStack.pop()
a = operandStack.pop()
counter.incrementPop()
counter.incrementPop()

result = calculate(a, op, b)
operandStack.push(result)
counter.incrementPush()
END FUNCTION

```

// ฟังก์ชันคำนวณค่าทางคณิตศาสตร์
FUNCTION calculate(a: Double, op: String,
b: Double) -> Double SWITCH op DO
    CASE "+": RETURN a + b
    CASE "-": RETURN a - b
    CASE "*": RETURN a * b
    CASE "/":
        IF b == 0 THEN
            THROW ArithmeticException("หารด้วยศูนย์")
        END IF
        RETURN a / b
    DEFAULT:
        THROW Exception("operator ไม่ถูกต้อง: " + op)
END SWITCH
END FUNCTION
// ฟังก์ชันกำหนดลำดับความสำคัญของตัวดำเนินการ
FUNCTION priority(op: String) -> Integer
    IF op == "+" OR op == "-" THEN RETURN 1
    IF op == "*" OR op == "/" THEN RETURN 2
    RETURN 0
END FUNCTION

END CLASS

```

```
Expression Processor:
1. Algorithm A (Infix -> Postfix -> Evaluate)
2. Algorithm B (Direct Infix Evaluate)
0. ออกจากโปรแกรม
เลือกเมนู: 2
ป้อนนิพจน์ (Infix): (8 + 2) * 5
--- Algorithm B ---
Postfix: (ไม่ได้สร้าง Postfix ใน Algorithm B)
ผลลัพธ์: 50.0
เวลาที่ใช้: 125000 ns
จำนวน Operation: Push=8, Pop=8, Comparisons=4, Loops: 7
```

ข้อดี

- ประมวลผลรอบเดียวจบ
- ประหยัดหน่วยความจำชั่วคราว
- คำนวณแบบ Real-time เมื่อพบตัวดำเนินการที่มีลำดับความสำคัญน้อยกว่าหรือพบวงเล็บปิด) ระบบจะดึง ข้อมูลมาคำนวณผลลัพธ์ย่อยทันที

ข้อจำกัด

- ตรรกะการทำงานซับซ้อน ต้องคุมการทำงานของ Stack 2 ชุดพร้อมกัน
- นำรูปแบบนิพจน์กลับมาใช้ซ้ำไม่ได้หากต้องการนำนิพจน์เดิมมาประมวลผลซ้ำ จะต้องวนอ่านและคำนวณโทเคน ใหม่ทั้งหมดตั้งแต่ต้น

Time Complexity

- $O(n)$

Space Complexity

- $O(n)$

3.ตัวอย่าง Step-by-step:

ตัวอย่างที่1 (กรณีปกติ) - Algorithm A: $3 + 4 * 2 / (1 - 5)$

Step	Token	Action	Operator Stack	Postfix สะสม
1	3	ตัวเลข -> ใส่ postfix	(ว่าง)	3
2	+	stack ว่าง -> push	+	3
3	4	ตัวเลข -> ใส่ postfix	+	3 4
4	*	priority(*) > priority(+) -> push	* +	3 4
5	2	ตัวเลข -> ใส่ postfix	* +	3 4 2
6	/	priority เท่ากับ * -> pop * ก่อน แล้ว push /	/ +	3 4 2 *

7	(	push เลย	(/ +	3 4 2 *
8	1	ตัวเลข -> ใส่ postfix	(/ +	3 4 2 * 1
9	-	top คือ (-> push เลย	- (/ +	3 4 2 * 1
10	5	ตัวเลข -> ใส่ postfix	- (/ +	3 4 2 * 1 5
11	)	pop จน เจอ (แล้ว ทิ้ง (	/ +	3 4 2 * 1 5-
12	(จบ)	pop ที่เหลือ ทั้งหมด	(ว่าง)	3 4 2 * 1 5 - / +

ผลลัพธ์Postfix:

- 3 4 2 * 1 5 - / + จากนั้นคำนวณ: $4 * 2 = 8$, $1 - 5 = -4$, $8 / -4 = -2$, $3 + (-2) = 1$ - ผลลัพธ์สุดท้ายคือ 1

ตัวอย่างที่2 (กรณีขอบเขต - วงเล็บซ้อน) - Algorithm B: $((8 + 2) * 5)$

Step	Token	Action	Operator Stack	Operand Stack
1	(	push	(	(ว่าง)
2	(	push	((	(ว่าง)
3	8	push operand	((	8
4	+	top คือ (-> push เลย	+ ((	8
5	2	push operand	+ ((	8 2
6	)	pop + คำนวณ $8 + 2 = 10$, push, pop (ทั้ง	(	10
7	*	top คือ (-> push เลย	* (	10
8	5	push	* (	10 5

9	)	pop * มา คำนวณ $10 * 5 =$ 50, push, pop (ทั้ง	(ว่าง)	50
10	(จบ)	operand stack เหลือตัว เดียว -> return	-	50

ผลลัพธ์:

= 50 (ตรวจสอบ manual: $(8 + 2) * 5 = 10 * 5 = 50$ ถูกต้อง) วงเล็บซ้อนกัน 2 ชั้นทำให้ operation stack มี (ค้างอยู่พร้อมกัน 2 ตัว และทุกครั้งที่เจอ) จะปิดการคำนวณของวงเล็บชั้นในสุดก่อนเสมอ ก็เลยสะท้อนว่า stack เหมาะกับปัญหานี้เพราะจัดการโครงสร้างที่ซ้อนกันแบบ LIFO ได้พอดี

4. อธิบายความถูกต้อง:

เลือก Algorithm A (InfixToPostfix) input $3 + 4 * 2 / (1 - 5)$

4.1 สภาพก่อนเริ่มอัลกอริทึม (Precondition)

- นิพจน์นำเข้า (Input Expression): " $3 + 4 * 2 / (1 - 5)$ " ถูกตัดคำ (Tokenize) ออกเป็น รายการสัญลักษณ์ (Token List) รวม 11 รายการ คือ ["3", "+", "4", "*", "2", "/", "(", "1", "-", "5", ")"]
- ตัวเก็บข้อมูล: สแต็กเก็บเครื่องหมายคำนวณ (Operator Stack), ส

สแต็กเก็บตัวเลข (Operand Stack), และ รายการผลลัพธ์โพสต์ฟิกซ์ (Postfix List) มีสถานะเป็นโครงสร้างข้อมูลว่างทั้งหมด

4.2 คุณสมบัติระหว่างทำงานที่ยังคงเป็นจริง (Loop Invariant)

- ช่วงแปลงเป็นโพสต์ฟิกซ์: ตัวเลขทุกตัวถูกส่งไปยัง รายการผลลัพธ์โพสต์ฟิกซ์(Postfix List) โดย คงลำดับจาก ซ้ายไปขวา เสมอ ส่วน สแต็กเก็บเครื่องหมายคำนวณ (Operator Stack) จะจัดเก็บเครื่องหมายเรียงตามลำดับ ความสำคัญ โดยเครื่องหมายที่มีความสำคัญสูงกว่าหรือเท่ากันจะถูกดึงออก (Pop) ไปก่อน
- ช่วงคำนวณโพสต์ฟิกซ์: ทุกครั้งที่เจอเครื่องหมาย ตัวเลข 2 ตัวล่าสุดใน สแต็กเก็บตัวเลข (Operand Stack) จะ ถูกดึงออก (Pop) มาคำนวณ แล้วนำผลลัพธ์ย่อยใส่กลับลง (Push) ใน สแต็กเก็บตัวเลข (Operand Stack) ทำให้ ค่าด้านบนสุดของ สแต็กเก็บตัวเลข (Operand Stack) เป็นผลลัพธ์ย่อยที่ต้องทำตามหลัก LIFO เสมอ

4.3 ผลลัพธ์เมื่ออัลกอริทึมหยุด (Postcondition)

- ผลลัพธ์ถูกต้อง: อัลกอริทึมคำนวณได้คำตอบสุดท้ายเท่ากับ 1.0 ตรงตามหลักคณิตศาสตร์
- เมื่อจบการทำงาน สแต็กเก็บตัวเลข (Operand Stack) เหลือตัวเลขเพียงตัวเดียวคือ 1.0 และ สแต็กเก็บ เครื่องหมายคำนวณ (Operator Stack) เป็นสแต็กว่าง

4.4 ข้อมูลที่ไม่เกี่ยวข้องสูญหายหรือเปลี่ยนลำดับหรือไม่

- ตัวเลข (Operands): อยู่ครบทั้ง 5 ตัว (3, 4, 2, 1, 5) และ ไม่มีการเปลี่ยนลำดับ จากซ้ายไปขวา - เครื่องหมายคำนวณ (Operators): อยู่

ครบทั้ง 4 ตัว (+, *, /, -) โดยถูกย้ายตำแหน่งใน รายการผลลัพธ์โพสต์ฟิกซ์(Postfix List) เพื่อกำหนดลำดับการคำนวณล่วงหน้า

4.5 อัลกอริทึมหยุดทำงานได้เสมอหรือไม่

- หยุดทำงานได้เสมอ 100%: รายการสัญลักษณ์(Token List) มีขนาดจำกัด เพียง 11 ตัว ($N = 11$)
- ลูปหลักทั้งสองขั้นตอนทำงานตามจำนวนสัญลักษณ์ใน รายการสัญลักษณ์(Token List) อย่างแน่นอน และการ ดึงข้อมูลออกจากสแต็ก ถูกจำกัดตามจำนวนองค์ประกอบที่เคยใส่ไว้โครงสร้างการทำงานจึงเป็น $O(N)$ และไม่มี ทางเกิด Infinite Loop

5. การวิเคราะห์ประสิทธิภาพ

Algorithm	Best Case	Average Case	Worst Case	Auxiliary Space	Total Space
A	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
B	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

6. โปรแกรมภาษา Java

Code อยู่ในลิ้ง github

<https://github.com/s68122250061-code/CSD2103SSRU/tree/720709a94bdb0083ed51e8c7029286e8cf030390/Group4>

```
Group4 > src > algorithms > J AlgorithmA.java > AlgorithmA
1  package algorithms;
2
3  import models.ExpressionResult;
4  import models.OperationCounter;
5  import utils.Tokenizer;
6
7  import java.util.ArrayDeque;
8  import java.util.ArrayList;
9  import java.util.Deque;
10 import java.util.List;
11
12 public class AlgorithmA {
13
14     public ExpressionResult evaluate(String expression) {
15         long startTime = System.nanoTime();
16         OperationCounter counter = new OperationCounter();
17
18         try {
19             List<String> tokens = Tokenizer.tokenize(expression);
20             List<String> postfix = infixToPostfix(tokens, counter);
21             double result = evaluatePostfix(postfix, counter);
22             String postfixStr = String.join(delimiter: " ", postfix);
23             long elapsedTime = System.nanoTime() - startTime;
24             return ExpressionResult.success(result, postfixStr, elapsedTime, counter);
25         } catch (Exception e) {
```

Expression Processor:

1. Algorithm A (Infix -> Postfix -> Evaluate)

2. Algorithm B (Direct Infix Evaluate)

0. ออกจากโปรแกรม

เลือกเมนู: 1

ป้อนนิพจน์ (Infix): 3 + 4 * 2 / (1 - 5)

--- Algorithm A ---

Postfix: 3 4 2 * 1 5 - / +

ผลลัพธ์: 1.0

เวลาที่ใช้: 1184200 ns

จำนวน Operation: Push=14, Pop=14, Comparisons=6, Loops: 20

```
Expression Processor:
1. Algorithm A (Infix -> Postfix -> Evaluate)
2. Algorithm B (Direct Infix Evaluate)
0. ออกจากโปรแกรม
เลือกเมนู: 2
ป้อนนิพจน์ (Infix): 3 + 4 * 2 / (1 - 5)
--- Algorithm B ---
Postfix: (ไม่ได้สร้าง Postfix ใน Algorithm B)
ผลลัพธ์: 1.0
เวลาที่ใช้: 2095200 ns
จำนวน Operation: Push-14, Pop-14, Comparisons-8, Loops: 11
```

ใช้ $3 + 4 * 2 / (1 - 5)$

การนับจำนวน Operation 14 เท่ากัน

การวัดเวลาในการทำงาน Algorithm A เร็วกว่า

****แต่ในทางทฤษฎีและการประมวลผลจริง Algorithm B มีประสิทธิภาพสูงกว่าและเร็วกว่า Algorithm A ****

7. Test Cases

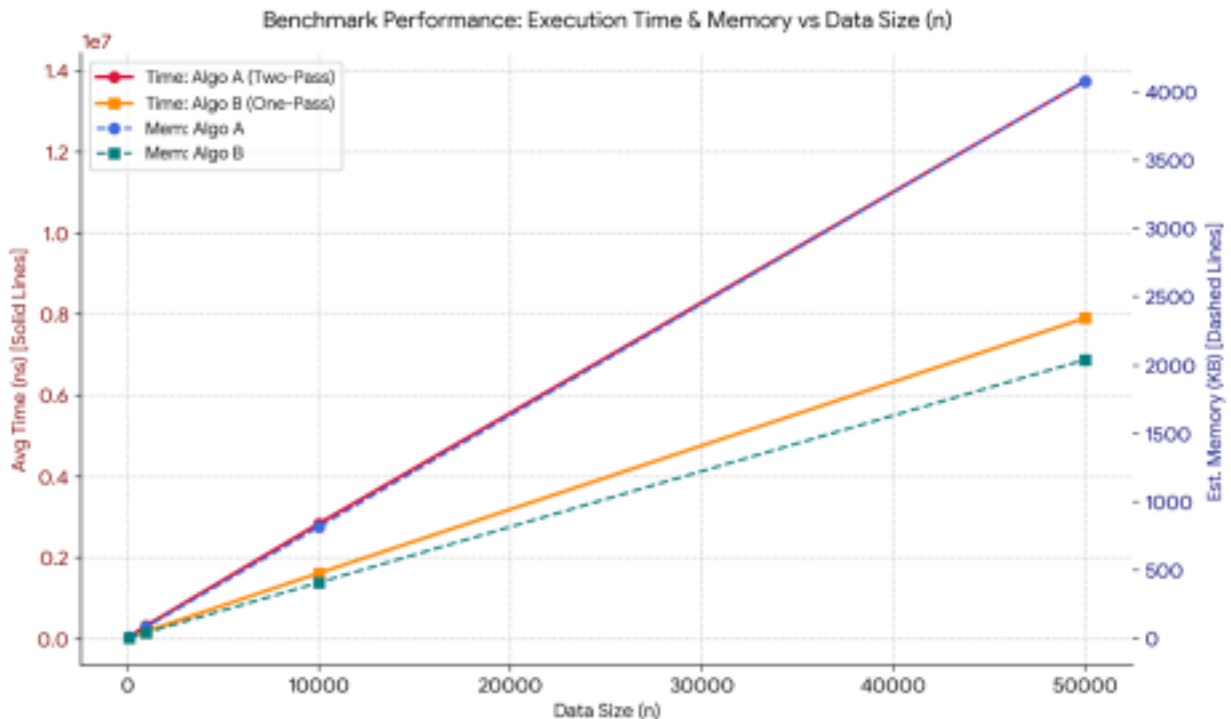
Test ID	Input	ประเภท	Expected Postfix	Expected Result
TC01	$3 + 4 * 2$	Normal case	$3\ 4\ 2\ *\ +$	11
TC02	$(3 + 4) * 2$	Normal case	$3\ 4\ +\ 2\ *$	14
TC03	$((8 + 2) * 5)$	Boundary (วงเล็บซ้อน)	$8\ 2\ +\ 5\ *$	50
TC04	$(3 + 4$	Invalid (วงเล็บไม่ครบ)	-	Error: วงเล็บไม่ครบคู่
TC05	$3 + 4)$	Invalid (วงเล็บเกิน)	-	Error: วงเล็บไม่ครบคู่
TC06	$3 + * 4$	Invalid (operator ซ้อน)	-	Error: นิพจน์ไม่ถูกต้อง

TC07	10 / (5 - 5)	Invalid (หารศูนย์)	10 5 5 - /	Error: หารด้วย ศูนย์
TC08	(นิพจน์ว่าง)	Empty input	-	Error: นิพจน์ ว่าง เปล่า
TC09	5	Best case	5	5
TC10	$3 + 4 * 2 / (1 - 5)$	Worst case	$3\ 4\ 2 * 1\ 5$ - / +	1

8. ผลการทดลองเปรียบเทียบประสิทธิภาพ

Size (n)	Algorithm	Avg Time (ns)	Push	Pop	Comparisons	Loop Iter	Data Moved	Est. Memory(KB)
100	Algorithm A	1,314,700	151	151	70	202	100	295.36
100	Algorithm B	416,220	151	151	71	101	0	89.77
1000	Algorithm A	3,302,640	1,501	1,501	767	2,002	1,000	1,104.85
1000	Algorithm B	1,488,400	1,501	1,501	769	1,001	0	865.96
10000	Algorithm A	17,914,500	15,001	15,001	7,539	20,002	10,000	3,710.69
10000	Algorithm B	8,640,600	15,001	15,001	7,540	10,001	0	1,650.52
50000	Algorithm A	45,440,320	75,001	75,001	37,527	100,002	50,000	44,642.17
50000	Algorithm B	8,900,200	75,001	75,001	37,528	50,001	0	30,725.30

PS E:\งานโปรแกรมนัฒน 4\Group4\src> □



อภิปรายว่าผลการทดลองสอดคล้องกับ Big-O หรือไม่

1. ความสอดคล้องกับ Big-O Notation

- Time Complexity $O(n)$: ผลการทดลองสอดคล้องกับทฤษฎี $O(n)$ อย่างสมบูรณ์เมื่อขนาดข้อมูล (n) เพิ่มขึ้น 10 เท่า (เช่น จาก $n = 1,000$ ไป $n = 10,000$) เวลาที่ใช้ในการประมวลผล (Average Time) และจำนวน ครั้งของปฏิบัติการพื้นฐาน (Push, Pop, Comparisons, Loop) ของทั้งสองอัลกอริทึมจะเพิ่มขึ้นในอัตราส่วน ประมาณ 10 เท่าตามเส้นตรง (Linear Scale)
- Space Complexity $O(n)$: ปริมาณหน่วยความจำที่ใช้ (Estimated Memory) ขยายตัวแปรผันตรงตามขนาด n แบบเชิงเส้นเช่นกัน

****การอธิบายความถูกต้อง กลุ่ม 4**

1. Operator ที่มีPriority สูงกว่าถูกประมวลผลก่อนอย่างไร
 - การ Pop Operator ที่มีPriority สูงกว่าหรือเท่ากันที่ค้างอยู่ใน Stack ออกไปใส่Postfix/คำนวณก่อนเสมอ ทำให้Operator ที่มีความสำคัญสูงกว่าถูกประมวลผลล่วงหน้าเสมอ
2. วงเล็บเปลี่ยนลำดับการคำนวณอย่างไร
 - เมื่อเจอ (อัลกอริทึมจะวาง Marker กัน priority ไว้ใน Stack ทำให้ Operator ข้างนอกไม่มาขัดจังหวะ จนกระทั่งเจอ) จึงจะ Pop ประมวลผลเฉพาะ Operator ภายในวงเล็บนั้นจนหมด
3. Postfix รักษาความหมายของ Infix อย่างไร
 - Postfix จัดความจำเป็นของวงเล็บและลำดับความสำคัญ โดยวาง Operand ไว้หน้า Operator เสมอ ทำให้ เรียงลำดับการคำนวณจากซ้ายไปขวาได้อย่างแม่นยำตรงตามนิพจน์ต้นฉบับ
4. Operand Stack เก็บผลลัพธ์ย่อยอย่างไร
 - นำ 2 ตัวบนสุดมาคำนวณ แล้ว Push ผลลัพธ์ย่อย (Sub-result) กลับลง Stack ทันทีเพื่อใช้เป็น Operand สำหรับ Operator ถัดไป

****คำถามวิเคราะห์กลุ่ม 4**

1. Balanced Parentheses มีComplexity เท่าใด

- Time Complexity เป็น $O(n)$ และ Auxiliary Space Complexity เป็น $O(n)$

2. Infix to Postfix มีComplexity เท่าใด

- Time Complexity เป็น $O(n)$ และ Auxiliary Space Complexity เป็น $O(n)$ เนื่องจากทุก Token ถูกอ่าน เพียงครั้งเดียว

3. เพราะเหตุใดแต่ละ Token จึงถูก Push และ Pop จำนวนจำกัด

- เพราะอัลกอริทึมทำงานแบบ Single-pass (อ่านจากซ้ายไปขวาแบบวนลูปรอบเดียว) Token แต่ละตัวจะถูก Push เข้า Stack เมื่อถึงลำดับของตอน และจะถูก Pop ออกเมื่อถึงเงื่อนไข Token 1 ตัว จะถูก Push อย่างมาก 1 ครั้ง และถูก Pop อย่างมาก 1 ครั้ง

4. Algorithm A และ B มีข้อดีและข้อเสียอย่างไร

Algorithm A

- ข้อดีคือ โครงสร้างโค้ดแยกส่วนชัดเจน (Modular) ตรวจสอบและ Debug รูปแบบ Postfix ได้ง่าย และสามารถ นำ Postfix ที่ได้ไปประมวลผลซ้ำได้โดยไม่ต้อง Parse ใหม่

- ข้อเสียคือ ต้องใช้ Memory เพิ่มขึ้นสำหรับเก็บ Postfix List และทำงาน 2 รอบ (Two-pass)

Algorithm B

- ข้อดีคือ ประหยัด Memory กว่าเพราะไม่ต้องเก็บ Postfix List และทำงานรอบเดียวจบ (Single-pass) - ข้อเสียคือ ซับซ้อนกว่า เพราะต้องจัดการ 2 Stack พร้อมกัน หากเกิดข้อผิดพลาดจะติดตามสถานะเพื่อ

Debug ได้ยากกว่า

5. วิธีใดตรวจสอบ Error ได้ง่ายกว่า

- Algorithm A ตรวจสอบ Error ได้ง่ายกว่า เนื่องจากแยกเฟสการตรวจสอบชัดเจน คือ เฟสไวยากรณ์(Syntax / Operator placement) ตอนแปลงเป็น Postfix และ เฟสคำนวณ (Evaluation / Division by zero / Operand ไม่พอ) ตอนประมวลผล Postfix ทำให้สามารถระบุตำแหน่งและสาเหตุของ Error ได้แม่นยำกว่า